

6 Nov 2023

Graphs ✓

1. How do we store graphs?
2. Examples of where graphs come up
3. terms related to graphs

A ^(directed) graph is a set V (where the elts are called vertices)
+ a set E of ^(directed) pairs of vertices.

digraph = directed graph

~~path~~
A walk is a sequence of vertices ^{such} that adjacent
verts in the walk are connected by a
(directed) edge.

A path is a walk that doesn't repeat
verts (other than possibly start/end)

A walk/path is closed if begins + ends at same
place.

A graph is acyclic if $\nexists$ a closed path
with 3+ vertices.

aka - acyclic = no nontrivial cycles.

other terms:

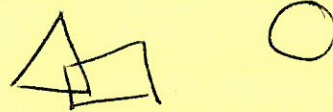
Weighted graphs = weights on edges
degree (of vertex), indegree, outdegree
traversals: BFS

Eulerian walk / tour - visit every edge exactly 1x
↑ linear-time algo exists $\Theta(n+m)$, $n=|V|$, $m=|E|$
Hamiltonian Path - visits every vertex exactly 1x
↑ NPC problem?

- trees • forests • spanning trees / forests
- connected • connected components

Examples

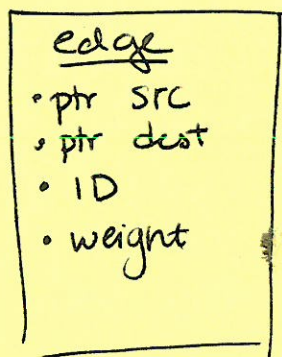
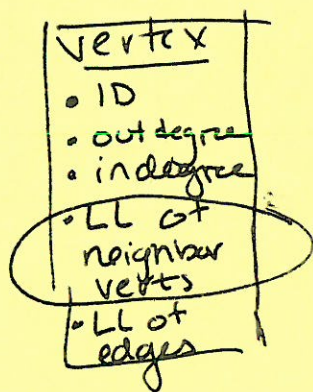
- networks (computer networks)
- binary search tree
- state diagrams
- game trees / graphs / phone diagram
- social networks
- road networks
- subway systems
- Contact graphs



- a node for each object
- an edge if they intersect

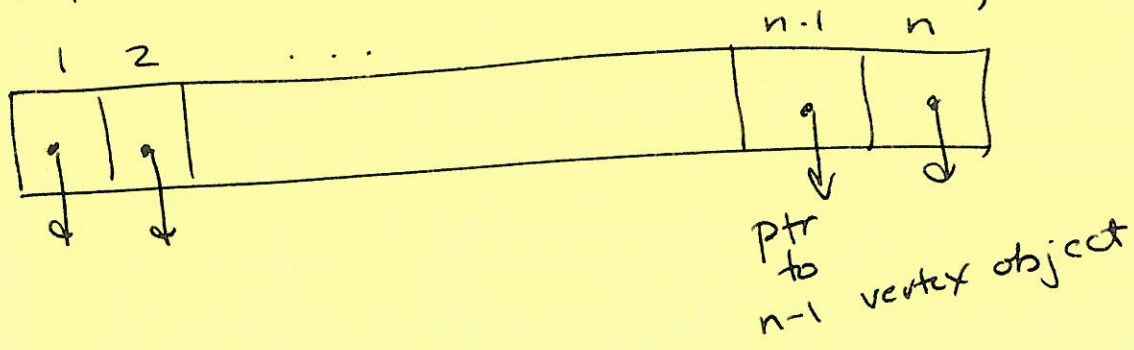
How to store graphs?

often, we have an object-oriented approach

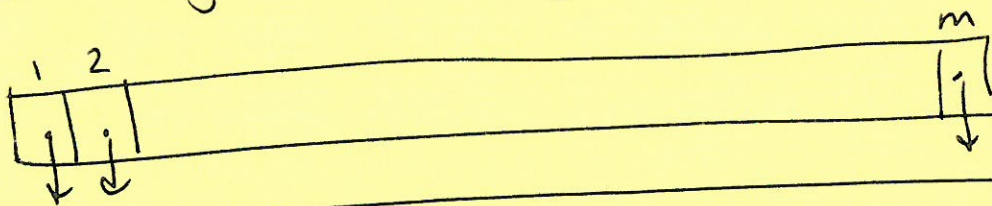


choose your own adventure. Might not need to store all of this info.

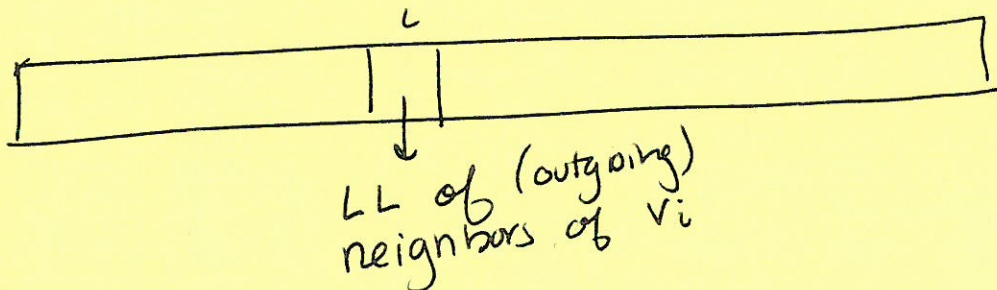
Often, we have a vertex array



and maybe an edgelist too

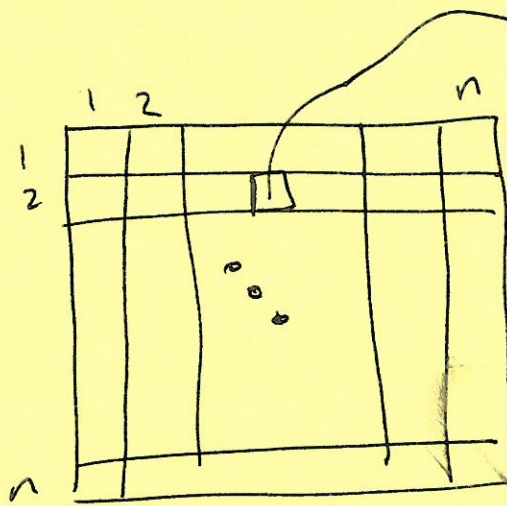


In this class, assume (unless otherwise stated) array of verts &



Note: assumes vertices # 1 through n

Adjacency Matrix $\rightarrow$ another common data structure



i, j entry store s :

- 0/1 to indicate if edge exists
- weight of edge
- ptr to an edge object or NULL

	Standard adjacency list (linked lists)	Fast adjacency list (hash tables)	Adjacency matrix
Space	$\Theta(V + E)$	$\Theta(V + E)$	$\Theta(V^2)$
Test if $uv \in E$	$O(1 + \min\{\deg(u), \deg(v)\}) = O(V)$	$O(1)$	$O(1)$
Test if $u \rightarrow v \in E$	$O(1 + \deg(u)) = O(V)$	$O(1)$	$O(1)$
List v 's (out-)neighbors	$\Theta(1 + \deg(v)) = O(V)$	$\Theta(1 + \deg(v)) = O(V)$	$\Theta(V)$
List all edges	$\Theta(V + E)$	$\Theta(V + E)$	$\Theta(V^2)$
Insert edge uv	$O(1)$	$O(1)^*$	$O(1)$
Delete edge uv	$O(\deg(u) + \deg(v)) = O(V)$	$O(1)^*$	$O(1)$

Table 5.1. Times for basic operations on standard graph data structures.

$$n = |V|$$

$$m = |E| \leq n(n-1) = \Theta(n^2)$$

~~if adjacency matrix has $\approx \Theta(n^2)$ non-zero entries,~~

if adjacency matrix has $\approx \Theta(n^2)$ non-zero entries, we call this a dense graph.

if adjacency matrix $\ll n^2$ non-zero entries, we call this a sparse graph.